

PROVISIONAL PATENT APPLICATION

Title of Invention

SYSTEM AND METHOD FOR A SELF-HEALING DETERMINISTIC RUNTIME

PIPELINE GOVERNING PROBABILISTIC AGENT EXECUTION

Inventor:

Leon Calvin Long II

Spicewood, Texas, United States

Entity Status: Small Entity / Micro Entity

Application Type: Provisional (35 U.S.C. § 111(b))

Filing Date: September 7, 2026

- 1 -

CROSS-REFERENCE TO RELATED APPLICATIONS

This application claims the benefit of and relates to the following co-pending applications of the same inventor, each of which is incorporated by reference in its entirety:

Number	Application Number	Title	Filed Date
1	64/119,191	Deterministically Governed Probabilistic Neural Computation	July 25, 2026
2	64/114,746	Universal Adaptive Intelligence Orchestration (Jasper Hypervisor)	July 18, 2026
3	64/094,360	Universal Rail to ISO 20022	June 19, 2026

Number	Application Number	Title	Filed Date
		Interoperability Bridge	
4	64/082,606	Cross-Ledger Data State Anchoring with Sequential Ordinal Satoshi Targeting	June 4, 2026
5	64/081,911	Universal File Reconstruction via Topological Geometric Calculus	June 3, 2026
6	64/081,490	Gas-Free Off-Chain Asset Transfer with Cryptographic Solvency Verification	June 3, 2026
7	64/145,825	Self-Healing AI Infrastructure with Chaos-Trained Neural Mesh	September 1, 2026

- 2 -

FIELD OF THE INVENTION

This invention relates generally to computer systems employing probabilistic computation (including large language models, neural networks, and autonomous software agents) in enterprise, financial, and infrastructure operations. More specifically, it relates to a deterministic runtime pipeline that validates, commits, moves, audits, and heals the outputs of probabilistic agents before those outputs may affect persistent state, financial systems, or third-party infrastructure, and to a healing loop that recovers from failures by replaying committed artifacts through deterministic layers without re-executing probabilistic intent parsing.

- 3 -

BACKGROUND

Probabilistic AI systems, including large language models, autonomous agents, and neural computational meshes, produce outputs that are inherently non-deterministic: the same input may yield different outputs across invocations. This non-determinism is unacceptable in enterprise and financial operations, where identical inputs must produce identical, auditable, and verifiable outcomes.

Prior approaches fail in three ways. First, attempts to make the probabilistic system itself deterministic, through constrained decoding and deterministic sampling, cannot eliminate the fundamental unpredictability of model behavior across contexts and versions. Second, middleware validation approaches check individual outputs but provide no commitment boundary, meaning no point at which a probabilistic proposal becomes deterministic, signed, immutable truth. Third, existing error-recovery approaches re-invoke the probabilistic system when a downstream failure occurs, re-rolling the original decision, converting a recovery operation into a new, unpredictable gamble, and destroying audit continuity between the original committed action and the recovered action.

The inventor's prior applications established a bifurcated governance doctrine whereby the mesh proposes and the runtime validates, a neural computational mesh governed by deterministic validation, and chaos-trained self-healing infrastructure. The present application discloses the complete deterministic runtime pipeline that operationalizes that doctrine: an eight-layer, forty-eight-control system through which every probabilistic agent output must pass, the cryptographic commitment boundary that converts probabilistic proposals into deterministic state, and the re-entry replay doctrine that permits self-healing without re-executing probabilistic intent.

SUMMARY OF THE INVENTION

A system and method comprising eight sequential layers through which every output of a probabilistic agent must pass before affecting persistent state:

1. Semantic Creation Layer: parses agent intent, resolves entities, constructs a weighted context graph, generates a semantic fingerprint, stamps source provenance, and gates proposals on an ambiguity score threshold. Proposals below the threshold are never auto-executed and are escalated for human review.
2. Canonicalization Layer: normalizes schemas and versions, deduplicates and conflict-merges fields, safely coerces types, standardizes encoding, enforces deterministic key ordering, and injects null-safety defaults.
3. Envelope Generation Layer: wraps the canonicalized payload with a structured metadata header, attaches a version tag and routing manifest, computes a checksum, cryptographically signs the envelope with a post-quantum signature (CRYSTALS-Dilithium3; RSA/ECDSA prohibited), assigns TTL and priority tier flags, and validates the envelope against a schema registry. The signed envelope constitutes a Commitment Boundary: the instant the envelope is signed, the probabilistic proposal has become deterministic, committed truth, and every downstream operation is deterministic.
4. Pre-Ingestion Validation Layer: verifies schema conformance, referential integrity across dependency chains, duplicate detection via a signature registry (bloom filter at fleet scale), authorization and scope verification with fail-closed tenant isolation (unfiltered tenant-scoped reads are rejected, never executed), rate-limit gate evaluation with back-pressure signaling, and a content-policy rule engine pass.
5. Continuity Substrate Layer: persists state with distributed checkpointing, maintains an idempotency key registry with TTL expiry (duplicate triggers exit silently rather than double-

execute), manages an ordered replay buffer, anchors a transaction log with write-ahead-log semantics and a first-class audit-trail completeness metric, enforces fault isolation boundaries (a single malformed record can never terminate a processing cycle), and coordinates cross-shard consistency with bidirectional drift detection and repair.

6. Universal Bridge Layer: translates protocols across heterogeneous targets (REST, gRPC, message queues, WebSockets, and blockchain settlement rails including Ethereum, Bitcoin, XRP, Solana, and Stellar), resolves cross-domain namespaces at the gate (never caller-supplied), performs load balancing with automatic failover, fans out and aggregates messages, injects observability hooks, and routes unrecoverable failures to a dead-letter queue with retry escalation.

7. Post-Movement Validation Layer: collects delivery receipts and acknowledgments (including human acknowledgment for critical operations), re-verifies payload integrity against the origin checksum, asserts SLA timing, audits side effects across downstream services, tests consistency convergence (strict for financial flows), and generates a state-diff reconciliation report.

8. Healing Loop Layer: detects anomalies, classifies root causes, selects retry strategy, dispatches compensating transactions and saga rollbacks, decides rollback versus forward-patch via a decision tree, escalates alerts with automated runbook triggers, and performs re-entry injection. Per the re-entry replay doctrine, which is the central novelty of this layer, re-entry replays the same committed signed envelope into Layer 3 or Layer 4. Intent is never re-parsed. A recovered operation therefore consumes the identical committed artifact as the original operation, preserving determinism and audit continuity. Re-entry into Layer 1, meaning re-parsing of probabilistic intent, is expressly prohibited by the system, as it would re-roll the probabilistic decision on what may be a financial operation.

Each successful healing event updates Pattern and Learning Metric records, which in turn adjust the gate thresholds of Layers 1 and 4, constituting a feedback loop by which the deterministic gates

are trained by the failures they intercept. New or modified healing playbooks are proposed through a self-improvement mechanism requiring human approval before deployment.

- 5 -

BRIEF DESCRIPTION OF THE DRAWINGS

FIG. 1 shows the eight-layer deterministic runtime pipeline, showing every probabilistic agent output entering at Layer 1, the commitment boundary at Layer 3, and the healing loop of Layer 8 feeding back to Layers 3 through 4.

FIG. 2 shows the structure of the Signed Envelope: payload, metadata header, version tag, routing manifest, checksum, post-quantum signature, TTL, and priority tier flags.

FIG. 3 shows the re-entry replay doctrine: a failed downstream operation triggering Layer 8 detection, remediation, and re-injection of the same signed envelope at Layer 3 or 4, with the intent re-parsing path shown as prohibited.

FIG. 4 shows the healing loop: anomaly detection, playbook matching with confidence gating, saga rollback and compensating transaction, verification, and pattern and learning update.

FIG. 5 shows the learning feedback: healing outcomes updating Pattern and LearningMetric records, which adjust Layer 1 ambiguity thresholds and Layer 4 gate policies.

FIG. 6 shows the dual-panel architecture: the probabilistic neural mesh (Ingest, Feature, Classify, Match, Heal, Learn) proposing, and the deterministic runtime pipeline validating, side by side.

Supporting exhibits: the inventor's published "Runtime Order Specification" (48 controls mapped one-to-one to implemented components, GitHub commit ba36fed, September 7, 2026) and "Runtime Era Dual-Panel Specification" (commit b63ab63, September 4, 2026) are attached as Exhibit A and Exhibit B respectively and incorporated by reference.

- 6 -

DETAILED DESCRIPTION OF THE PREFERRED EMBODIMENTS

Layer 1 — Semantic Creation (Probabilistic Front-End)

Every action proposed by a probabilistic agent enters the pipeline here. An intent parser and disambiguation engine converts natural-language or structured agent intent into a typed proposal. An entity-resolution step tags all referenced entities and their relationships. A context graph is constructed with weighted edges, the weights deriving from the operational neural mesh (per application 64/119,191). A semantic fingerprint, being an embedding-class vector, identifies the proposal's type for pattern matching. Source provenance and lineage are stamped at creation, such that every downstream artifact carries a verifiable chain of custody back to its originating agent, timestamp, and session.

The layer terminates in an Ambiguity Score Threshold Gate: proposals whose confidence score falls below a configurable threshold (operationally 0.80) are held in a detected state and escalated for human review. They may never auto-execute. In the reduction to practice, this gate has exercised on 20 or more anomalies per chaos-training run with zero false positives.

Layer 2 — Canonicalization

The proposal is normalized into a canonical schema: version alignment across heterogeneous entity definitions, field deduplication with conflict-merge resolution, type coercion with safe casting (wrong types are rejected at the boundary rather than coerced silently where lossy), UTF-8 encoding standardization, deterministic key ordering (which enables reliable idempotency signatures downstream), and null-safety enforcement with honest-null semantics. Fields representing unmeasured results remain null until measured; defaults are injected only where semantically safe.

Layer 3 — Envelope Generation (The Commitment Boundary)

The canonical payload is wrapped into an envelope comprising: (a) a structured metadata header; (b) a version tag and routing manifest declaring the envelope's destination and processing path; (c) a SHA-256 checksum of the payload; (d) a cryptographic signature applied using CRYSTALS-Dilithium3 (post-quantum; embodiments may use Kyber-1024 for key encapsulation and SPHINCS+-256f where applicable; classical RSA/ECDSA signatures are excluded by policy); (e) a time-to-live assignment and priority tier flags; and (f) validation against an envelope schema registry.

The signing of the envelope is the Commitment Boundary of the system. Before signing, the artifact is a probabilistic proposal that may be freely discarded. After signing, the artifact is deterministic committed truth: identical on every replay, verifiable by any party holding the public verification key, and immutable. All subsequent layers operate deterministically on committed artifacts.

Layer 4 — Pre-Ingestion Validation

Six gates, all of which must pass: (1) schema conformance and structural integrity; (2) referential integrity across the linked-record dependency chain (anomaly, healing event, playbook, and ticket); (3) duplicate detection via a deterministic signature registry, wherein every workflow computes a deterministic signature (workflow identity plus execution window) and consults the registry before executing, and a duplicate signature causes silent exit, such that double-fired triggers never double-execute, never double-count learning data, and never double-bill; (4) authorization scope verification with fail-closed tenant isolation, wherein an unfiltered read or write of a tenant-scoped record is rejected rather than executed, with the rejection itself audited; (5) rate-limit gate evaluation with back-pressure signaling via a circuit-breaker state machine; and (6) a content-policy rule engine enforcing constitutional constraints on agent behavior, including

prohibition of financial execution by agents lacking express grant, prohibition of external webhook invocation, and immutability of the runtime's governance rules.

Layer 5 — Continuity Substrate

State persistence with distributed checkpointing; an idempotency key registry with TTL expiry; an ordered replay buffer permitting full reconstruction of operational history; transaction log anchoring with write-ahead-log semantics, where audit-trail completeness is tracked as a first-class monitored metric; fault isolation boundaries such that a single malformed record cannot terminate a processing cycle (per-record isolation with error capture); and a cross-shard consistency coordinator performing bidirectional drift detection between stores, with automatic drift repair on a scheduled cadence and silent operation when consistent.

Layer 6 — Universal Bridge

Protocol translation across heterogeneous targets, including REST, gRPC, message queues, WebSockets, and public blockchain settlement rails (Ethereum, Bitcoin, XRP, Solana, and Stellar) through per-rail adapters behind a single interface. Cross-domain namespace resolution (tenant and customer identifiers) is resolved at the gate from session context, never caller-supplied. Load balancing with automatic failover retains degraded-mode fallbacks during proof periods for higher-priority channels. Message fan-out fans a single committed event to multiple observation and action surfaces, with fan-in aggregation collecting results into reconciliation records. Observability hooks (heartbeats and telemetry events) are injected at every stage. Unrecoverable messages route to a dead-letter queue with retry escalation terminating in human-visible alerting.

Layer 7 — Post-Movement Validation

Delivery receipt confirmation and acknowledgment collection, including for critical-severity operations human acknowledgment recorded as a precondition to resolution. Payload integrity re-

verification against the origin checksum. SLA timing assertion against configurable first-response and resolution policies. Side-effect audit scanning across downstream services before an operation may close. Consistency convergence testing, being strict consistency for financial transaction flows and eventually-consistent convergence for operational flows, with state-diff reconciliation report generation.

Layer 8 — Healing Loop and the Re-Entry Replay Doctrine

Anomaly detection with root-cause classification; retry strategy selection including exponential backoff and jitter; compensating transaction dispatch and saga rollback for multi-step operations; a rollback-versus-forward-patch decision tree (forward-patch where the committed envelope is sound and the failure was environmental; rollback where the envelope itself fails verification); alert escalation with automated runbook (playbook) triggering; and re-entry injection.

The re-entry replay doctrine is the central novelty of this layer. Upon remediation, the system re-injects the original signed envelope, being the identical committed artifact, into Layer 3 (Envelope Generation, for re-validation and re-signing of the identical payload) or Layer 4 (Pre-Ingestion Validation, for re-gating). The system never re-executes Layer 1 intent parsing on a re-entry. Consequences: (a) the recovered operation is bit-identical in intent to the original operation, meaning no probabilistic re-roll; (b) audit continuity is preserved, as the recovery is provably the same action, not a new one; and (c) a failed financial operation cannot mutate into a different financial operation during recovery.

In the reduction to practice, the chaos-training regime exercises this doctrine continuously: synthetic anomalies are injected into a live fleet, remediated through playbooks, and re-entered via envelope replay, with auto-heal ratios, gating ratios, and escalation ratios tracked cycle-over-cycle as learning metrics.

Loop closure: every successful healing event updates Pattern and LearningMetric records; these records adjust Layer 1 ambiguity thresholds and Layer 4 gate policies, so the deterministic gates are progressively trained by the failures they intercept. New or modified healing playbooks are proposed through a self-improvement mechanism and require human approval before deployment.

- 7 -

METHOD OF CONSTRUCTION AND OPERATION

In addition to the pipeline system disclosed above, the inventor discloses the overall method by which a governed, self-healing probabilistic computing infrastructure is constructed, deployed, and operated, being the method practiced across the inventor's production fleet:

Step 1 — Deterministic Substrate Provisioning. A persistent, schema-validated entity state store is provisioned with typed records and automatic audit fields (identity and timestamps), such that all persistent state is validated at write time and fully attributed.

Step 2 — Deterministic Cage Construction. Each probabilistic agent (LLM-based) is instantiated with a capability-scoped configuration validated by a deterministic configuration gate: capabilities, status, and integration grants are checked against a schema and an allowlist, and a configuration whose prompt promises a refusal the grants contradict is rejected with zero-violation pass required. Agents therefore enter operation only with grants exactly matching their declared behavioral contract.

Step 3 — Bifurcated Operation. Agents propose actions through the eight-layer pipeline of this application; the deterministic runtime validates, commits, executes, audits, and heals. No probabilistic output touches persistent state, financial systems, or third-party infrastructure except through the pipeline.

Step 4 — Chaos-Training of the Deterministic Gates. Synthetic anomalies are injected into the live production infrastructure on a cyclical ramp (200 to 1000 injections per run, step-wise, cyclically repeating), with a novelty rule (a minimum fraction of novel anomaly types per run, increased at cycle start) to prevent overfitting. An auto-heal ratio (healed versus gated versus escalated) is tracked run-over-run as a first-class learning metric. The gates and playbooks are thereby trained by the failures they intercept, in production, rather than in simulation.

Step 5 — Learning Feedback into the Mesh. Every healing event updates Pattern and LearningMetric records; the records adjust gate thresholds (Step 3) and the weights of a neural computational mesh, in which a large language model performs the role of a compute engine executing forward and backward propagation over an entity-persisted network topology (per application 64/119,191). The mesh thereby learns from real operational outcomes.

Step 6 — Hypervisor Supervision Across Application Instances. A single deterministic supervisor governs a plurality of independently instantiated applications via typed signal routing, dispatch graphs with fallback chains, epoch lifecycle state machines, and a hash-chained ledger of governance decisions (per application 64/114,746), escalating critical events to a human operator through a dedicated escalation channel.

Step 7 — Template Instantiation. The method is distributed as an immutable template package: a reference entity set, a playbook library, operational skills, and a neural node topology instantiated into a host application, after which heartbeat monitoring, anomaly detection, and healing operate in the host under hypervisor supervision without modification of the host's domain logic.

Step 8 — Platform-Agnostic Adaptation. Healing intents are expressed independently of platform, executed through platform adapters (Microsoft/Azure/M365, iOS, Windows, and macOS), and telemetry is ingested back through the pipeline, such that the method operates across heterogeneous host platforms.

The combination of Steps 1 through 8 constitutes the inventor's method of constructing and operating a self-healing, deterministically governed probabilistic computing infrastructure, as reduced to practice across 37 or more production applications and 500 or more healing events.

- 8 -

REDUCTION TO PRACTICE

The invention is reduced to practice and operating in production across a fleet of 37 or more deployed software applications, including: a deterministic agent factory validating agent configurations against capability allowlists (zero-violation gate), a governance-cage chat agent whose proposal, approval, and execution functions are separately gated (money-execution refused by design), a multi-rail settlement bridge executing the Layer 6 translation live across five blockchains, a bidirectional cross-store consistency coordinator (twice-daily drift repair), and a continuous chaos-training lifecycle (200-to-1000 injection ramp, cyclical, with novelty rules preventing overfitting). Benchmarks: 267x anomaly-lookup speedup (800 milliseconds to 3 milliseconds), 25 active mesh nodes across 10 agents communicating collision-free, zero false positives across 70 or more stress-test anomalies and the full chaos lifecycle.

- 9 -

INFORMAL CLAIMS

Claim 1. A method of governing probabilistic agent outputs, comprising: receiving a proposed action from a probabilistic agent; gating the proposed action on an ambiguity threshold; canonicalizing the gated proposal; generating a signed envelope comprising the canonicalized proposal, a checksum, and a post-quantum signature, wherein the signing constitutes a commitment boundary after which all processing is deterministic; validating the signed envelope

through a pre-ingestion gate stack; and executing the committed envelope only upon passage of all gates.

Claim 2. The method of claim 1, wherein a failed execution triggers a healing loop that replays the same signed envelope into the validation layers without re-executing probabilistic intent parsing.

Claim 3. The method of claim 1, wherein the pre-ingestion gate stack rejects, in a fail-closed manner, any tenant-scoped operation lacking a tenant identifier resolved from authenticated session context.

Claim 4. The method of claim 1, further comprising an idempotency signature registry wherein duplicate execution attempts exit silently.

Claim 5. The method of claim 2, wherein outcomes of healing events update learning records that adjust the ambiguity threshold and the gate stack policies.

Claim 6. A method of constructing a self-healing probabilistic computing infrastructure, comprising: provisioning a schema-validated entity state store; instantiating probabilistic agents with capability-scoped configurations validated by a deterministic configuration gate; operating the agents in a propose-only relationship to a deterministic validation pipeline; chaos-training the pipeline's gates by cyclic synthetic anomaly injection into the live infrastructure with a novelty rule and an auto-heal ratio metric; feeding healing outcomes into pattern and learning records that adjust both the gates and the weights of an entity-persisted neural mesh; supervising a plurality of application instances under a single deterministic hypervisor with escalation to a human operator for critical events; and distributing the whole as a template instantiated into host applications without modifying host domain logic.

ABSTRACT

A self-healing deterministic runtime pipeline governing probabilistic agent execution comprising eight sequential layers, being Semantic Creation, Canonicalization, Envelope Generation, Pre-Ingestion Validation, Continuity Substrate, Universal Bridge, Post-Movement Validation, and Healing Loop, through which every probabilistic agent output must pass before affecting persistent state or financial systems. A cryptographic commitment boundary at Layer 3 converts probabilistic proposals into immutable, signed, deterministic artifacts using post-quantum signatures. A re-entry replay doctrine in Layer 8 permits self-healing by replaying the same committed signed envelope without re-executing probabilistic intent parsing, preserving audit continuity and preventing mutation of recovered financial operations. A learning feedback loop trains the deterministic gates from healing outcomes, adjusting ambiguity thresholds and gate policies cycle-over-cycle. The system is reduced to practice across 37 or more production applications with 500 or more healing events, zero false positives across all chaos-training runs, and a 267x anomaly-lookup performance improvement.